Hummingbird API Debugging Assignment

Submission Report

Student: Mayank Tamakuwala
Date: March 7, 2026
Assignment: Debug the Hummingbird API using Claude Code
Environment: AWS CloudShell, us-west-2, Claude Code v2.1.71, AWS Bedrock (Opus 4.6)
Status: All 4 tickets + Bonus completed and verified

Project Context

What is Hummingbird?

Hummingbird is a media management REST API built with Node.js and Express. It allows users to upload images, track their processing status, and download processed versions. The system runs on AWS infrastructure provisioned via Terraform.

Architecture Overview

```
Client --> ALB (port 80)
             |
             v
       ECS Fargate (port 9000) --> Express API
             |
             +---> S3 (uploads/ and resized/)
             +---> DynamoDB (PK=MEDIA#id, SK=METADATA)
             +---> SNS --> SQS --> Worker (ECS)
```

Infrastructure components:

- **ALB:** Receives client traffic on port 80, forwards to ECS on port 9000
- **ECS Fargate:** Runs the API container and a background worker
- **S3:** Stores uploads under `uploads/<mediaId>/<filename>` and processed copies under `resized/<mediaId>/<filename>`

- **DynamoDB:** Single table with composite key (PK + SK). Each media item uses `PK=MEDIA#<mediaId>` and `SK=METADATA`
- **SNS/SQS:** Event bus for async processing. Upload triggers `media.v1.resize` via SNS, fanning out to SQS

Image Lifecycle

1. **Upload (POST /v1/media/upload?width=N):** Streams file to S3, saves metadata to DynamoDB with `status=PENDING`, publishes resize event to SNS, returns `202 { mediaId }`.
2. **Background Processing:** Worker polls SQS, sets status to `PROCESSING`, copies file from `uploads/` to `resized/` in S3, sets status to `COMPLETE`. Also available synchronously via `PUT /v1/media/:id/resize`.
3. **Status Polling (GET /v1/media/:id/status):** Returns `{ status }` so clients know when processing is done.
4. **Download (GET /v1/media/:id/download):** If `COMPLETE`, returns `302` redirect to presigned S3 URL. If not ready, returns `202` with `Retry-After` and `Location` headers.

Key Source Files

File	Purpose
`server.js`	Express setup, health check, mounts routes
`routes/media.js`	Route definitions with middleware
`controllers/media.js`	Request handlers (upload, status, download, get, resize, delete)
`actions/uploadMedia.js`	Streaming upload to S3 via formidable
`clients/s3.js`	S3 operations (upload, presign URL, copy, delete)
`clients/dynamodb.js`	DynamoDB CRUD (createMedia, getMedia, setMediaStatus)
`clients/sns.js`	Publishes resize/delete events to SNS
`worker/processor.js`	SQS consumer for resize and delete events
`core/constants.js`	Status enums, event types, file size limits
`core/responses.js`	HTTP response helpers

Investigation Methodology

For each ticket:

1. Read the source code using Claude Code
2. Reproduce the bug with curl against the live ALB
3. Check CloudWatch logs
4. Identify the root cause
5. Apply the fix
6. Rebuild and redeploy via `docker build` + `aws ecs update-service`
7. Verify the fix with another curl request

Ticket #1: Server Crashes on Startup Without APP_PORT

Severity: Critical
File: `server.js`
Line: 35
Category: Configuration / Environment Variable Handling

Bug Description

The Express server reads the listening port from the `APP_PORT` environment variable but provides no fallback default. If the variable is missing, `process.env.APP_PORT` evaluates to `undefined`. Node.js then attempts `app.listen(undefined, ...)`, which either crashes or binds to a random port. The ALB health check cannot reach the container on the expected port.

Detailed Investigation

The server startup code at `server.js:35` was:

```
const port = process.env.APP_PORT;

app.listen(port, () => {
  logger.info(
    `Example app listening on port ${port}`
  );
});
```

The infrastructure expects port 9000, confirmed in Terraform:

1. `terraform/variables.tf:25-29` defines the default:

```
variable "app_port" {
  description = "Port the app listens on"
  type        = number
  default     = 9000
}
```

2. `terraform/ecs.tf:65` injects it into the container:

```
environment = [
  {
    name  = "APP_PORT",
    value = tostring(var.app_port)
  }
]
```

3. `terraform/ecs.tf:58-63` maps the container port:

```
portMappings = [
  {
    containerPort = var.app_port,
    hostPort      = var.app_port,
    protocol      = "tcp"
  }
]
```

4. `terraform/alb.tf:15` configures the target group:

```
port     = var.app_port
protocol = "HTTP"
```

When the ECS task definition is correctly configured, `APP_PORT=9000` is injected and everything works. But if the variable is ever missing (local development, task definition misconfiguration, running outside ECS), the server fails silently.

Root Cause

Missing JavaScript logical OR (`||`) default value operator. The developer assumed `APP_PORT` would always be set by the infrastructure, but defensive coding requires a fallback.

Code Diff

Before (broken):

```
const port = process.env.APP_PORT;
```

After (fixed):

```
const port = process.env.APP_PORT || 9000;
```

Full Fixed Code Block (server.js:32-38)

```
app.use('/v1/media', mediaRoutes);

const port = process.env.APP_PORT || 9000;

app.listen(port, () => {
  logger.info(
    `Example app listening on port ${port}`
  );
});
```

Why This Fix Is Correct

- Fallback `9000` matches Terraform default (`variables.tf:27`)
- ALB target group routes to port 9000 (`alb.tf:15`)
- ECS port mapping uses port 9000 (`ecs.tf:60`)
- `||` only activates when `APP_PORT` is falsy
- In ECS with proper env vars, fallback is never used

Impact Without Fix

- Container starts on undefined port or crashes
- ALB health checks fail (expecting port 9000)
- ECS marks task as unhealthy, restarts in a loop
- Service becomes completely unavailable

Ticket #2: Width Field Missing from GET Response

Severity: High
File: `clients/dynamodb.js`
Line: 84 (inside `getMedia` function)
Category: Data Retrieval / Incomplete Response

Bug Description

When a client uploads an image with a specific `width` parameter (e.g., `?width=800`), the width is correctly saved to DynamoDB. However, when retrieving metadata via `GET /v1/media/:id`, the width field is missing. The `getMedia()` function reads the DynamoDB record but omits the `width` attribute from its return value.

Detailed Investigation

Step 1: Trace the upload path.

`controllers/media.js:27` (`uploadController`) calls `createMedia()` in `clients/dynamodb.js:23`, which saves:

```
// clients/dynamodb.js:23-36 (createMedia)
const command = new PutItemCommand({
  TableName,
  Item: {
    PK: { S: `MEDIA#${mediaId}` },
    SK: { S: 'METADATA' },
    size: { N: size.toString() },
    name: { S: name },
    mimetype: { S: mimetype },
    status: { S: MEDIA_STATUS.PENDING },
    width: { N: String(width) },  // saved
  },
});
```

Step 2: Trace the retrieval path.

`GET /v1/media/:id` calls `getMedia()` in `clients/dynamodb.js:56`, which returns:

```
// clients/dynamodb.js:78-84 (BEFORE fix)
return {
  mediaId,
  size: Number(Item.size.N),
  name: Item.name.S,
  mimetype: Item.mimetype.S,
  status: Item.status.S,
  // width is NOT included here
};
```

The width attribute exists in `Item.width.N` but was never mapped into the return value.

Evidence from Live API

```
# Upload with width=800
$ curl -X POST "$ALB/v1/media/upload?width=800" \
    -F "file=@sample.png"

Response:
{"mediaId":"89f6a00d-281b-426e-88d5-609c421866d0"}

# Retrieve metadata - width is missing
$ curl "$ALB/v1/media/89f6a00d-281b-426e-88d5-609c421866d0"

Response:
{
  "mediaId": "89f6a00d-281b-426e-88d5-609c421866d0",
  "size": 700210,
  "name": "sample.png",
  "mimetype": "image/png",
  "status": "PENDING"
}

PROBLEM: No "width" field, even though upload used width=800
```

Root Cause

The developer who wrote `createMedia` remembered to persist the width field to DynamoDB, but `getMedia` forgot to include it in the return object. This is a classic "write-read asymmetry" bug: data is stored correctly but not retrieved completely.

Code Diff

Before (broken):

```
  return {
    mediaId,
    size: Number(Item.size.N),
    name: Item.name.S,
    mimetype: Item.mimetype.S,
    status: Item.status.S,
  };
```

After (fixed):

```
  return {
    mediaId,
    size: Number(Item.size.N),
    name: Item.name.S,
    mimetype: Item.mimetype.S,
    status: Item.status.S,
    width: Number(Item.width.N),
  };
```

Full Fixed Function (clients/dynamodb.js:56-87)

```
  async function getMedia(mediaId) {
    try {
      const command = new GetItemCommand({
        TableName,
        Key: {
          PK: { S: `MEDIA#${mediaId}` },
          SK: { S: 'METADATA' },
        },
      });

      logger.info({ mediaId, sk: 'METADATA' });

      const client = new DynamoDBClient({
        region: process.env.AWS_REGION,
      });
```

```
    const { Item } = await client.send(command);

    if (!Item) {
      return null;
    }

    return {
      mediaId,
      size: Number(Item.size.N),
      name: Item.name.S,
      mimetype: Item.mimetype.S,
      status: Item.status.S,
      width: Number(Item.width.N), // ADDED
    };
  } catch (error) {
    logger.error(error);
    throw error;
  }
}
```

Why This Fix Is Correct

- `Item.width.N` accesses the DynamoDB Number attribute saved by `createMedia`
- `Number()` converts the DynamoDB string representation back to a JavaScript number, matching the pattern used for `size`
- The field name `width` matches what `createMedia` stores and what the middleware parsed from the request
- This makes `getMedia` symmetrical with `createMedia`

Impact Without Fix

- Clients cannot retrieve the resize width they requested
- Downstream logic depending on width breaks
- API contract is inconsistent: upload accepts width, GET doesn't return it
- Violates the principle of least surprise

Ticket #3: Location Header Missing HTTP Protocol Prefix

Severity: High
File: `controllers/media.js`
Line: 111
Category: HTTP Standards Compliance / URL Construction

Bug Description

When the download endpoint returns a `202 Accepted` response (media still processing), it includes a `Location` header pointing to the status endpoint. However, the header is constructed without the `http://` protocol prefix, producing an invalid URL.

Detailed Investigation

Step 1: Read the download controller.

The `downloadController` at `controllers/media.js:98-130` handles `GET /v1/media/:id/download`:

```
// controllers/media.js:108-119 (BEFORE fix)
if (media.status !== MEDIA_STATUS.PROCESSING) {
  const SIXTY_SECONDS = 60;
  res.set('Retry-After', SIXTY_SECONDS);
  res.set(
    'Location',
    `${req.hostname}/v1/media/${mediaId}/status`
  );
  // ... returns 202
}
```

Step 2: Understand the Express API difference.

Express provides two ways to access the request host:

Express Method	Returns	Example
`req.hostname`	Domain only, port stripped	`example.com`
`req.get('host')`	Full Host header with port	`example.com:9000`

The code used `req.hostname`, which:

1. Strips the port number
2. Does not include a protocol scheme

Step 3: Understand the HTTP specification.

Per RFC 7231, the `Location` header should be an absolute URI:

- Valid: `http://example.com/v1/media/abc/status`
- Invalid: `example.com/v1/media/abc/status` (missing scheme)

Evidence from Live API

Before fix:

```
$ curl -i "$ALB/v1/media/<id>/download"

HTTP/1.1 202 Accepted
Retry-After: 60
Location: hummingbird-production-alb-1860718987
  .us-west-2.elb.amazonaws.com
  /v1/media/<id>/status

PROBLEM: Missing http:// prefix!
```

After fix:

```
$ curl -i "$ALB/v1/media/<id>/download"

HTTP/1.1 202 Accepted
Retry-After: 60
Location: http://hummingbird-production-alb
  -876316285.us-west-2.elb.amazonaws.com
  /v1/media/<id>/status

FIXED: Valid absolute URL with http://
```

Root Cause

The developer used `req.hostname` instead of `req.get('host')` and forgot to prepend the protocol scheme. This is a common Express.js mistake because `req.hostname` sounds like it should return everything needed for a URL, but it only returns the bare domain.

Code Diff

Before (broken):

```
res.set(
  'Location',
  `${req.hostname}/v1/media/${mediaId}/status`
);
```

After (fixed):

```
res.set(
  'Location',
  `http://${req.get('host')}/v1/media/${mediaId}/status`
);
```

Why This Fix Is Correct

- `req.get('host')` returns the full Host header including port
- Prepending `http://` produces a well-formed absolute URI
- Behind ALB, the Host header reflects the ALB's DNS name
- In local dev on port 9000, it correctly returns `localhost:9000`

Impact Without Fix

- HTTP clients that follow Location headers fail or misroute
- Automated retry logic in client SDKs breaks
- Violates RFC 7231 and industry conventions for 202 responses

---

Ticket #4: Download Endpoint Never Serves Completed Files

Severity: Critical
File: `controllers/media.js`
Line: 108
Category: Business Logic / State Machine Error

Bug Description

The download endpoint (`GET /v1/media/:id/download`) should return a `302` redirect to a presigned S3 URL when processing is complete. Instead, it always returns `202 Accepted` regardless

of status, because the comparison logic is backwards.

Detailed Investigation

Step 1: Understand the status state machine.

Defined in `core/constants.js:22` :

```
const MEDIA_STATUS = {
  PENDING: 'PENDING',
  PROCESSING: 'PROCESSING',
  COMPLETE: 'COMPLETE',
  ERROR: 'ERROR',
};
```

Only `COMPLETE` means the file is ready to serve.

Step 2: Read the buggy condition.

```
// controllers/media.js:108 (BEFORE fix)
if (media.status !== MEDIA_STATUS.PROCESSING) {
```

This reads: "If status is anything other than PROCESSING, return 202."

Trace through each status:

Status	!== PROCESSING	Result	Correct?
PENDING	true	202	Yes
PROCESSING	false	302 redirect	WRONG
COMPLETE	true	202	WRONG
ERROR	true	202	Acceptable

The logic is inverted. It serves the file only when PROCESSING (actively being worked on) and blocks it when COMPLETE (actually ready).

Step 3: Reproduce the bug.

```
# Upload an image
$ curl -X POST "$ALB/v1/media/upload?width=500" \
    -F "file=@sample.png"
{"mediaId":"a67a4aec-383f-48ec-8d0c-39bd27460225"}
```

```
# Synchronously resize (sets status to COMPLETE)
$ curl -X PUT \
    "$ALB/v1/media/a67a4aec-.../resize?width=500"
{"status":"COMPLETE"}

# Try to download - should be 302, gets 202
$ curl -i "$ALB/v1/media/a67a4aec-.../download"
HTTP/1.1 202 Accepted
Retry-After: 60
{"message":"Media processing in progress."}

BUG: Returns 202 even though status is COMPLETE
```

Root Cause

The developer confused which status to gate on. Instead of checking "is the file NOT complete?" they checked "is the file NOT processing?" The condition should guard against incomplete states, allowing only `COMPLETE` to pass through.

Code Diff

Before (broken):

```
  if (media.status !== MEDIA_STATUS.PROCESSING) {
```

After (fixed):

```
  if (media.status !== MEDIA_STATUS.COMPLETE) {
```

Corrected Status Flow

Status	!== COMPLETE	Result	Correct?
PENDING	true	202	Yes
PROCESSING	true	202	Yes
COMPLETE	false	302 redirect	Yes
ERROR	true	202	Acceptable

Full Fixed downloadController (controllers/media.js:98-130)

```
async function downloadController(req, res) {
  try {
    const { id: mediaId } = req.params;

    const media = await getMedia(mediaId);
    if (!media) {
      return sendNotFoundResponse(
        res, 'Media not found'
      );
    }

    // FIXED: was !== PROCESSING
    if (media.status !== MEDIA_STATUS.COMPLETE) {
      const SIXTY_SECONDS = 60;
      res.set('Retry-After', SIXTY_SECONDS);
      // FIXED: was req.hostname (no protocol)
      res.set(
        'Location',
        `http://${req.get('host')}/v1/media/${mediaId}/status`
      );
      logger.info(
        { mediaId, currentStatus: media.status },
        'Media not ready for download, sending 202'
      );
      return sendAcceptedResponse(res, {
        message: 'Media processing in progress.'
      });
    }

    const url = await getProcessedMediaUrl({
      mediaId,
      mediaName: media.name,
    });

    return res.redirect(302, url);
  } catch (error) {
    logger.error(error);
    return sendInternalServerErrorResponse(res);
  }
}
```

Design Pattern: Async Processing Status Gates

When implementing download endpoints for async resources:

1. Gate on the **terminal success state** (COMPLETE), not an intermediate one
2. All non-success states should return a retry response (202 with Retry-After)
3. Use `status !== SUCCESS` (inclusive deny), not `status !== INTERMEDIATE`
4. Consider handling ERROR explicitly with a 4xx/5xx

Impact Without Fix

- No client can ever download a completed file
- The entire download flow is broken
- The presigned S3 URL code (lines 122-125) is unreachable
- Users upload files but can never retrieve them

BONUS: DynamoDB Sort Key Case Mismatch

Severity: Critical (Data Integrity)
File: `clients/dynamodb.js`
Lines: 155, 167
Category: Database Key Consistency / Silent Failure

Bug Description

The `setMediaStatus()` function uses a lowercase Sort Key value `'metadata'`, while `createMedia()` and `getMedia()` use uppercase `'METADATA'`. Because DynamoDB keys are case-sensitive, status updates are written to a completely different (phantom) item. The original item's status remains `PENDING` forever.

Detailed Investigation

Step 1: Map all DynamoDB key usage.

Three functions in `clients/dynamodb.js` interact with the same DynamoDB record:

Function	Line	SK Value	Operation
createMedia	29	`METADATA` (uppercase)	PutItem

Function	Line	SK Value	Operation
getMedia	62	`METADATA` (uppercase)	GetItem
setMediaStatus	155	`metadata` (lowercase)	UpdateItem

The SK values don't match.

Step 2: Understand DynamoDB key behavior.

DynamoDB treats Sort Key values as raw binary data. String comparisons are byte-level, meaning `METADATA` and `metadata` are entirely different keys pointing to different items.

When `setMediaStatus` sends an `UpdateItemCommand` with `SK: { S: 'metadata' }`:

- DynamoDB looks for `PK=MEDIA#<id>` and `SK=metadata`
- No such item exists (real item has `SK=METADATA`)
- DynamoDB's default UpdateItem behavior is upsert
- It silently creates a phantom item with `SK=metadata`
- The original item with `SK=METADATA` is untouched

Step 3: Confirm with CloudWatch logs.

The logger calls conveniently log the SK value:

```
# createMedia uses uppercase
{"mediaId":"5457fc51-...","sk":"METADATA"}

# setMediaStatus uses lowercase (WRONG ITEM!)
{"mediaId":"5457fc51-...",
 "newStatus":"PROCESSING","sk":"metadata"}

# Second update also lowercase (WRONG ITEM!)
{"mediaId":"5457fc51-...",
 "newStatus":"COMPLETE","sk":"metadata"}
```

The first log shows `"sk":"METADATA"` (from createMedia), but subsequent logs show `"sk":"metadata"` (from setMediaStatus). This is the smoking gun.

Step 4: Reproduce the cascading failure.

```
# Upload an image
$ curl -X POST "$ALB/v1/media/upload?width=500" \
    -F "file=@sample.png"
{"mediaId":"5457fc51-..."}
```

```
# Trigger resize - claims COMPLETE
$ curl -X PUT \
    "$ALB/v1/media/5457fc51-.../resize?width=500"
{"status":"COMPLETE"}

# Check actual status - still PENDING!
$ curl "$ALB/v1/media/5457fc51-.../status"
{"status":"PENDING"}
```

The resize endpoint returns `"status":"COMPLETE"` because it's hardcoded in `resizeController` (line 169) — it doesn't read back from DynamoDB. But `getMedia` reads the original item (`SK=METADATA`) which was never updated.

Step 5: Visualize the data corruption.

After the resize, DynamoDB contains TWO items:

PK	SK	status	Other fields
MEDIA#5457fc51...	METADATA	PENDING	size, name, etc.
MEDIA#5457fc51...	metadata	COMPLETE	(only status)

Row 1 is the real item (from createMedia). Row 2 is the phantom (from setMediaStatus upsert). All reads go to row 1; all updates go to row 2.

Root Cause

A typo or inconsistency in the Sort Key value. The developer used lowercase `'metadata'` in `setMediaStatus` while the rest of the codebase uses uppercase `'METADATA'` . Since there's no constant reference for the SK value (raw string literals in each function), this mismatch went undetected.

Code Diff

Change 1 - UpdateItemCommand Key (line 155):

```
    Key: {
      PK: { S: `MEDIA#${mediaId}` },
-     SK: { S: 'metadata' },
+     SK: { S: 'METADATA' },
    },
```

Change 2 - Logger message (line 167):

```
- logger.info(
-   { mediaId, sk: 'metadata', newStatus },
-   'Updating media status in DynamoDB'
- );
+ logger.info(
+   { mediaId, sk: 'METADATA', newStatus },
+   'Updating media status in DynamoDB'
+ );
```

Full Fixed Function (clients/dynamodb.js:148-171)

```
async function setMediaStatus({
  mediaId,
  newStatus
}) {
  try {
    const command = new UpdateItemCommand({
      TableName,
      Key: {
        PK: { S: `MEDIA#${mediaId}` },
        SK: { S: 'METADATA' }, // FIXED
      },
      UpdateExpression:
        'SET #status = :newStatus',
      ExpressionAttributeNames: {
        '#status': 'status'
      },
      ExpressionAttributeValues: {
        ':newStatus': { S: newStatus },
      },
    });

    const ddbClient = new DynamoDBClient({
      region: process.env.AWS_REGION,
    });

    logger.info(
      { mediaId, sk: 'METADATA', newStatus },
      'Updating media status in DynamoDB'
    ); // FIXED
```

```
      await ddbClient.send(command);
    } catch (error) {
      logger.error(error);
      throw error;
    }
  }
```

Why This Bug Is Particularly Dangerous

1. **No error signals:** DynamoDB doesn't throw an error when updating a non-existent item — it silently creates one. No error logs, no exceptions, no failed health checks.
2. **Misleading responses:** The resize endpoint returns `"status":"COMPLETE"` because it's hardcoded in the controller (not read from DynamoDB), giving a false impression.
3. **Log-level detection only:** The only way to spot it is comparing `"sk"` values across log lines. There's no single error line.
4. **Data integrity violation:** The database contains orphaned phantom items that are never read.
5. **Cascading with Ticket #4:** Even after fixing the status comparison, downloads still fail because `getMedia` reads the original item (stuck at PENDING). Both bugs must be fixed together.

How This Should Be Prevented

- **Use constants:** Define `const SORT_KEY = 'METADATA'` once and reference everywhere
- **Write integration tests:** A test that creates, updates, and reads back a record catches this immediately
- **Add ConditionExpression:** Use `attribute_exists(PK)` to make UpdateItem fail instead of upsert

---

Combined Impact Analysis

The five bugs interact to create a completely broken download flow:

```
Ticket #1 (Port):
  APP_PORT missing -> server doesn't start

Ticket #2 (Width):
  Upload works, but clients can't see width
```

```
Bonus (SK mismatch):
  Status updates go to phantom DynamoDB item
  Real item stays PENDING forever

Ticket #4 (Status gate):
  Even if status were COMPLETE, the condition
  checks the wrong value and blocks download

Ticket #3 (Location header):
  Even the "retry later" response gives
  clients an invalid URL to poll
```

After all fixes:

- Server starts reliably on port 9000
- Upload saves and returns all metadata including width
- Status updates target the correct DynamoDB item
- Download correctly identifies COMPLETE and redirects
- The retry Location header is a valid absolute URL

Summary of All Fixes

#	File	Line(s)	Bug	Fix
1	server.js	35	No fallback port	Added `|| 9000`
2	clients/dynamodb.js	84	Missing width	Added `width: Number(Item.width.N)`
3	controllers/media.js	111	No protocol in URL	Use `http://${req.get('host')}`
4	controllers/media.js	108	Wrong comparison	Changed to `!== COMPLETE`
B	clients/dynamodb.js	155, 167	SK case mismatch	Changed to uppercase `METADATA`

Technical Learnings

1. Defensive Configuration

Always provide fallback values for critical environment variables. Infrastructure-as-code defines defaults, but application code should also be resilient. Use `process.env.VAR || default`.

2. Write-Read Symmetry

When persisting data to a database, ensure every field that is written is also read back. Review write and read paths together. A mismatch means data is stored but inaccessible.

3. HTTP Header Construction in Express.js

Express provides multiple ways to access host information:

- `req.hostname` strips the port and returns only the domain
- `req.get('host')` returns the full Host header including port
- Neither includes the protocol scheme — always prepend manually
- For production, consider `req.protocol` for the scheme

4. State Machine Design

When implementing async processing with status polling:

- Gate on the terminal success state (COMPLETE), not intermediate
- All other states should return a retry or error response
- Write the condition as `status !== SUCCESS` to be inclusive

5. DynamoDB Key Consistency

DynamoDB keys are binary-compared and case-sensitive. A single case difference creates an entirely different item. Best practices:

- Define key values as constants and reference everywhere
- Never use raw string literals for keys in multiple functions
- Use ConditionExpression to prevent silent upserts
- Write integration tests for the full create-update-read cycle

6. Silent Failures Are the Most Dangerous

The bonus bug produced zero error logs. DynamoDB silently created a phantom item. The resize endpoint hardcoded its response. Everything appeared to work, but data was split across two items. Design systems to fail loudly when invariants are violated.